

# INPUT/OUTPUT DATA STREAM

## CONTENTS OF THE CHAPTER

- *Data stream.*
- *Input/output with binary data.*
- *Input/output with text data.*
- *Working with object data.*

*Input/output operations are crucial in programming. Java strongly supports this by providing the java.io package, which contains classes for working with various data sources. This allows programmers to easily create programs to work with files and other data sources such as keyboards, screens, printers, network data, etc.*

## 7.1. DATA STREAM

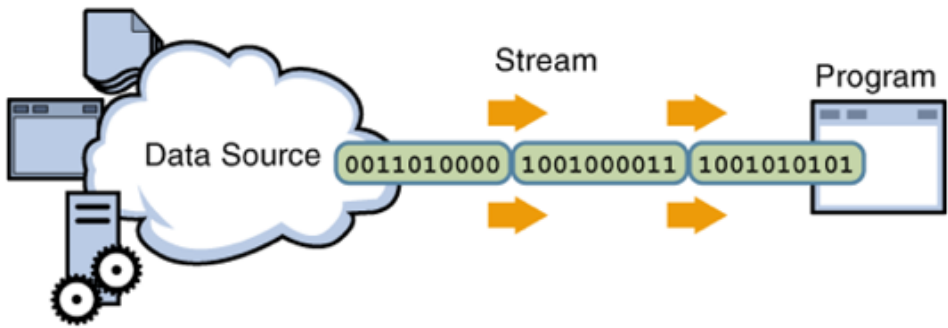
### 7.1.1. The concept of data flow

Input/output in Java is stream-based:

- An object from which we can READ is called an input stream.
- An object from which we can WRITE is called an output stream.

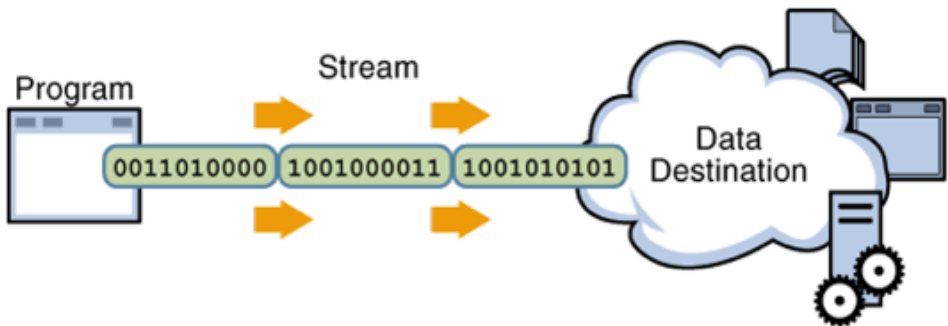
A stream represents a data flow, or a communication channel, with a writer component at one end and a reader component at the other.

Input/output streams are used to work with files and communicate over networks. They can also be used to communicate between two streams running simultaneously and have stream classes for reading and writing data stored in the computer's memory. A program can read information from multiple sources such as a file system, a server, or another program that needs an input data stream:



**Figure 7-1. Input data flow**

Conversely, a program that outputs a stream of data will create a data source for storage in the file system, a storage server, or another application:



**Figure 7-2. Output data stream**

### 7.1.2. Character and binary data

There are two common types of data used in input/output: binary data (defined and readable by computers) and text (readable by humans). Binary data is data encoded into binary bits; it can represent numbers, characters, data structures, or multimedia data such as images and audio. Essentially, binary data can represent almost any type of data in a computer. Human-readable data is in character form. When you read a number, such as 3.141592654, you are reading a sequence of characters and interpreting them as a number. This number can also be represented in binary form, which humans cannot (or find very difficult to) read.

To work with these two types of data representation, Java has two corresponding stream types: character streams and binary streams (object

streams can be considered a special case of binary streams). There are several built-in classes in the Java library that represent streams of each type.

Byte streams can be useful for direct communication between computers and for storing most data formats. However, binary data is often difficult, or rather impossible, to read by humans. When faced with a long string of zeros and ones, a reader will not know what information it represents or how it is encoded without knowing its structure. The current trend seems to be towards increased use of character data, expressed in a way that makes its meaning as clear as possible.

Example number 127:

**Table 7-1. Comparison table of text data and binary data**

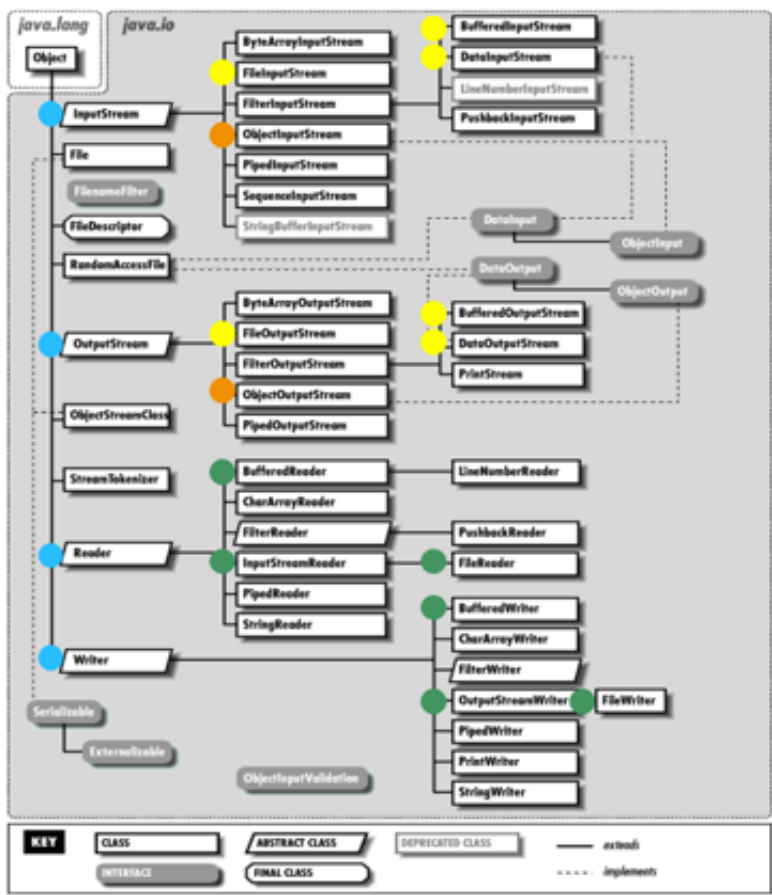
| Text data: '1', '2', '7'        | Binary data: 127                                      |
|---------------------------------|-------------------------------------------------------|
| ASCII (Hex): 31, 32, 37         | 1 byte (byte): 01111110                               |
| Unicode (Hex): 0031, 0032, 0037 | 2 bytes (short): 00000000 01111110                    |
| UTF8 (Hex): 31, 32, 37          | 4 bytes (int): 00000000 00000000<br>00000000 01111110 |

**7.2. JAVA.IO PACKAGE**

Java defines classes for input/output in the java.io package. The java.io package contains a full suite of classes for generating input and output streams. An object for reading binary data would be a subclass of the abstract InputStream class, while an object for outputting data to a binary stream would be one of the subclasses of the abstract OutputStream class.

To read and write human-readable character data, the main classes are the abstract Reader and Writer classes. All character stream classes are subclasses of one of these. If a number is written to a Writer stream, the computer must translate it into a human-readable string representing that number. Characters are stored in the computer as 16-bit Unicode values. For those using the English alphabet, character data is typically stored in files using ASCII codes, which only use 8 bits per character. The Reader and Writer classes handle this translation and can also handle alphabets and

characters from many countries.

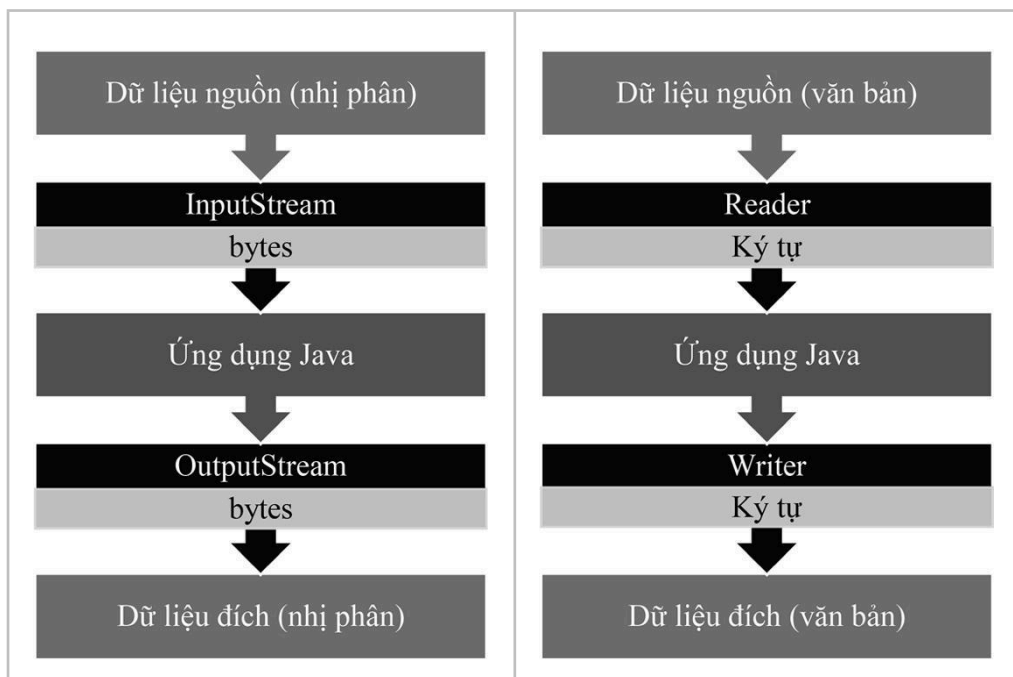


When using classes in the java.io package to read and write data, the main steps to follow are as follows:

1. import java.io.\*;
2. Open a new line;
3. Use the stream to perform read or write operations as required.
4. Close that line.

It can be seen that the Input/Output Stream and Reader/Writer classes provide methods for working with streams. Depending on the data to be manipulated, different classes will be used; this chapter will discuss the following classes:

- Input and output streams are used to read and write binary data:
  - `FileInputStream` and `FileOutputStream`.
  - `DataInputStream` and `DataOutputStream`.
  - `BufferedInputStream` and `BufferedOutputStream`.
- Readers and writers are used to read and write text data.
- `InputStreamReader` and `OutputStreamWriter`.
- `InputStreamReader` and `OutputStreamWriter`.
- `FileReader` and `FileWriter`.



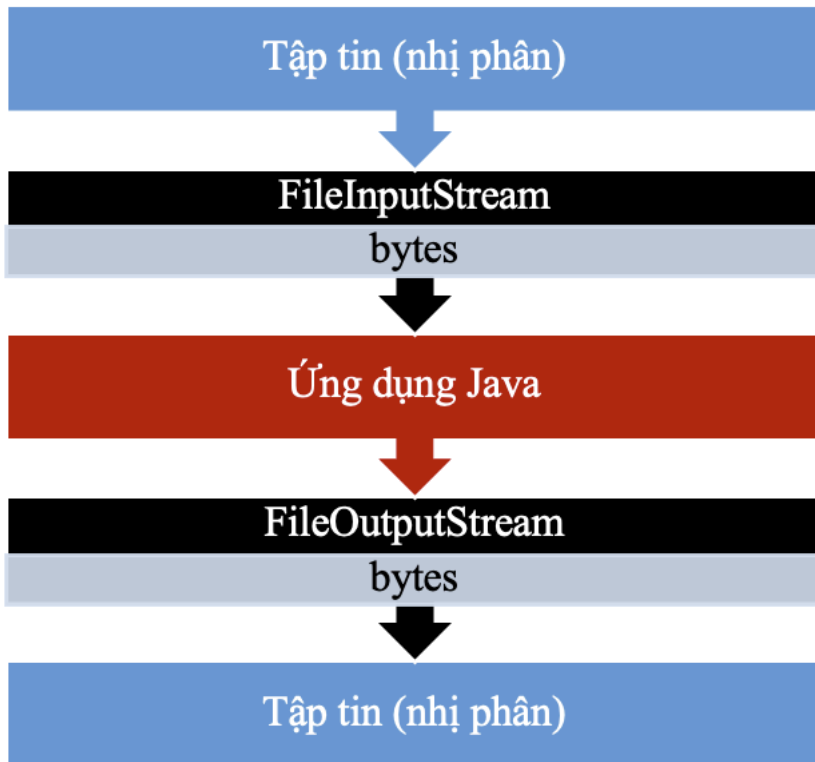
## 7.3. BINARY DATA STREAM

### 7.3.1. `FileInputStream` and `FileOutputStream`

Reading and writing files are two frequently used operations in application programming. These two operations can be easily performed using `FileInputStream` and `FileOutputStream`:

- **`FileInputStream`** reads data bytes from a file.

- **FileOutputStream** is used to write data byte by byte to a file.



and **DataOutputStream** read/write methods

| Recording methods<br>of <b>FileOutputStream</b> | Reading methods<br>of <b>FileOutputStream</b> | Describe                          |
|-------------------------------------------------|-----------------------------------------------|-----------------------------------|
| write()                                         | read()                                        | Read and write one or more bytes. |

For example, to copy an image file, you can read each byte of the original file and copy it to the new file. The input parameter of the **FileInputStream** constructor is the file path:

```
try {
    FileInputStream fin = new FileInputStream( "SourcePics.jpg"
);
```

```

    FileOutputStream fout = new FileOutputStream(
"DestPics.jpg" );
    int c ;
    while (( c = fin .read()) != - 1 )
        fout.write ( c );
    if ( fin != null )
        fin.close ();
    if ( fout != null )
        fout.close ();
} catch (IOException e) {
System.out.println ( "An error occurred while reading/writing
the file " ) ;
}

```

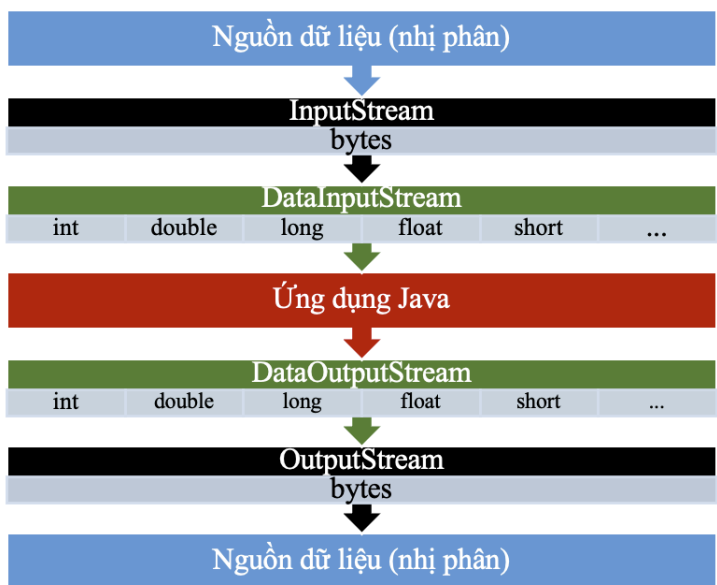
Note that when working with Streams, constructors and read/write methods often throw "checked" exceptions, so try-catch blocks are always necessary when writing code. To catch specific errors, pay attention to the following exception types:

- **FileNotFoundException** : An exception thrown by constructors when a line to a file cannot be created.
- **IOException**: An exception thrown during data reading/writing from a stream.

However, it can be seen that the objects created from `FileInputStream` and `FileOutputStream` only provide simple functions that allow reading one or more bytes. This is useful for raw data consisting of bytes, where the bytes are read sequentially. However, for structured files, manipulating data using these classes will be difficult.

### 7.3.2. `DataInputStream` and `DataOutputStream`

**`DataInputStream`** class provides read methods that correspond to the write methods in **`DataOutputStream`** for reading and writing underlying data.



The methods of this class are shown in the table below:

**and DataOutputStream read and write methods**

| Recording methods of DataOutputStream            | DataInputStream read methods                  | Describe                                                    |
|--------------------------------------------------|-----------------------------------------------|-------------------------------------------------------------|
| writeByte(),writeShort(), writeInt(),writeLong() | readByte(), readShort(), readInt(),readLong() | Read and write integers: byte, short, int, long             |
| writeFloat(), writeDouble()                      | readFloat(), readDouble()                     | Read and write floating-point numbers (float, double).      |
| planChar()                                       | readChar()                                    | Read and write a char character.                            |
| writeBoolean()                                   | readBoolean()                                 | Read and write a boolean data point.                        |
| writeUTF()                                       | readBytes(), readUTF()                        | Read and write a string of characters, or a Unicode string. |
| writeChars()                                     | But                                           | Write multiple ch                                           |



|             |     |                      |
|-------------|-----|----------------------|
|             |     | characters           |
| writeByte() | But | Write multiple bytes |

The constructor of the `DataOutputStream` class requires passing in `OutputStreams`. For example, if you want to write an integer to a file:

```
DataOutputStream dos = new DataOutputStream( new
FileOutputStream( "Int.txt" ));
int i = 100 ;
dos.writeInt(i);
dos.close();
```

Write code to read the above data from the file:

```
FileInputStream fis = new FileInputStream( "Int.txt" );
DataInputStream dis = new DataInputStream(fis);
int i = dis.readInt();
System.out.println ( i);
dis.close();
```

The result will be displayed on the screen:

100

### 7.3.3. `BufferedInputStream` and `BufferedOutputStream`

Buffered streams add a buffering mechanism that allows for more efficient input/output. When a buffered stream is created, a buffer is also created. This buffer helps to transfer data, resulting in faster data processing speeds.



and `DataOutputStream` read and write methods

| BufferedOutputStream<br>recording methods | BufferedInputStream<br>reading methods | Describe                          |
|-------------------------------------------|----------------------------------------|-----------------------------------|
| write()                                   | read()                                 | Read and write one or more bytes. |

Similar to `DataInputStream` and `DataOutputStream`, the constructors of `BufferedOutputStream` and `BufferedInputStream` are input/output streams. For example:

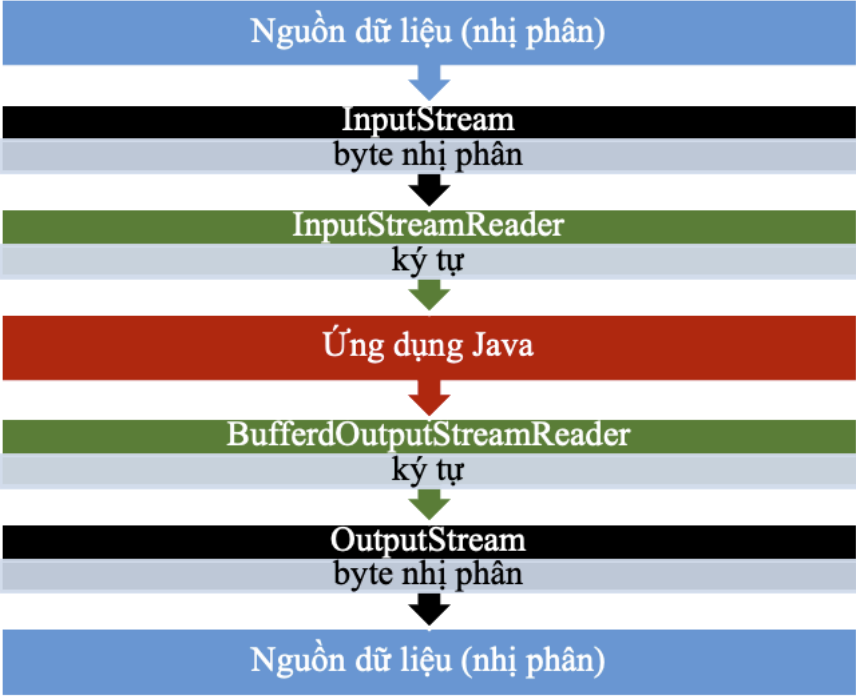
```
BufferedInputStream bis = new BufferedInputStream( new
FileInputStream( "a.pdf" ));
BufferedOutputStream bos = new BufferedOutputStream( new
FileOutputStream( "b.pdf" ));
```

## 7.4. TEXT DATA STREAMS

### 7.4.1. InputStreamReader and OutputStreamWriter

`InputStreamReader` and `OutputStreamWriter` function similarly to

other input streams, but they convert data from binary to character format for easier text manipulation. The converted characters can be encoded using a specific character encoding scheme, with UTF being the most common today.



The OutputStreamWriter and InputStreamReader classes provide the following methods:

**Table 7-5. Read and write methods of OutputStreamWriter and InputStreamReader**

| Recording methods<br>of OutputStreamWriter | Reading methods<br>of InputStreamReader | Describe                                         |
|--------------------------------------------|-----------------------------------------|--------------------------------------------------|
| write()                                    | read()                                  | Read and write one or more characters to a line. |

The constructors for both of these classes will require the corresponding InputStream and OutputStream.

The following example demonstrates writing code to write text (UTF8

character encoding) to a file, then reading the file back and displaying its contents on the screen:

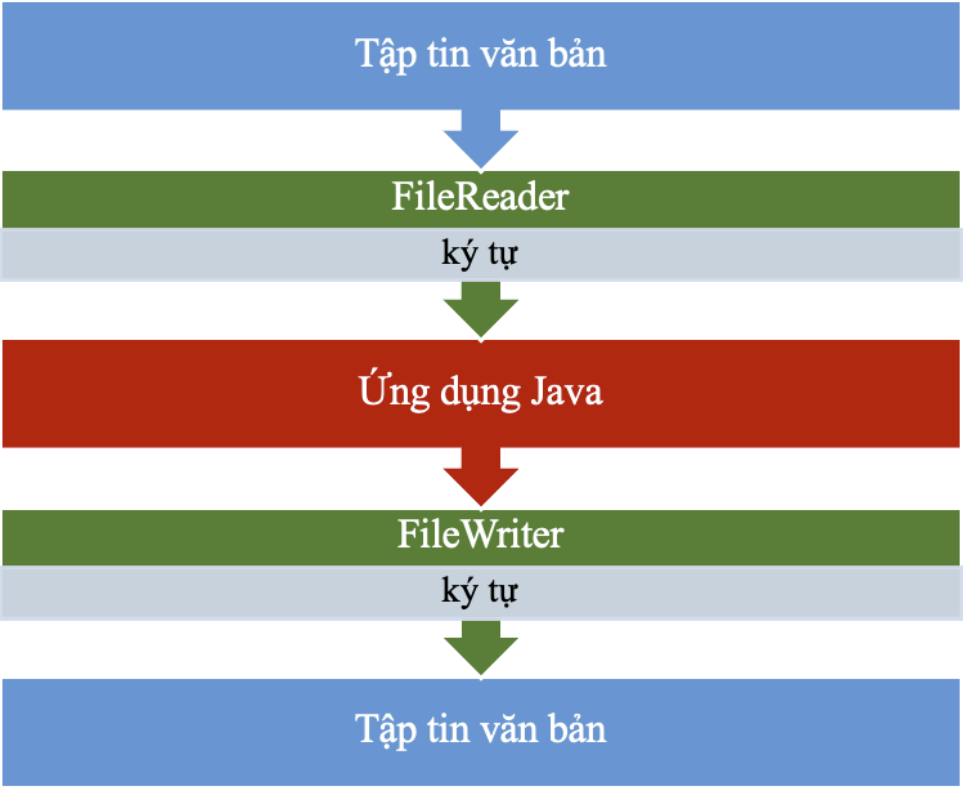
```
try {
    FileOutputStream fos = new FileOutputStream( "TestUTF8.txt"
);
    OutputStreamWriter out = new OutputStreamWriter( fos ,
"UTF8" );
    out .write("Hello, Vietnam");
    out.close ();
    FileInputStream fis = new FileInputStream( "TestUTF8.txt"
);
    Reader in = new InputStreamReader( fis , "UTF8" );
    int charRead;
    while (( charRead = in .read()) != - 1 ) {
        System . out .printf( "%c(%02X) " , ( char ) charRead
, charRead );
    }
    in .close();
} catch (IOException ex) {
ex.printStackTrace();
}
```

The results are displayed on the screen:

```
H(48) e(65) l(6C) l(6C) o(6F) ,(2C) (20) V(56) i(69) ê(1EC7)
t(74) (20) N(4E) a(61) m(6D)
```

#### 7.4.2. FileReader and FileWriter

FileReader and FileWriter are classes that make working with text files easier. Their constructors allow you to directly pass the file name to generate a line of text.



FileReader and FileWriter also provide write() and read() methods to read and write characters to files, similar to InputStreamReader and OutputStreamWriter.

**Table 7-6. FileWriter and FileReader read and write methods**

| Recording methods<br>of FileWriter | Reading methods<br>of FileReader | Describe                                         |
|------------------------------------|----------------------------------|--------------------------------------------------|
| write()                            | read()                           | Read and write one or more characters to a line. |

For example, write text data to the file text.txt using FileWriter.

```
FileWriter out = new FileWriter ( "text.txt" );
out.write( "FileReader" );
out.flush();
out.close();
```

Read text data from the aforementioned text file using FileReader.

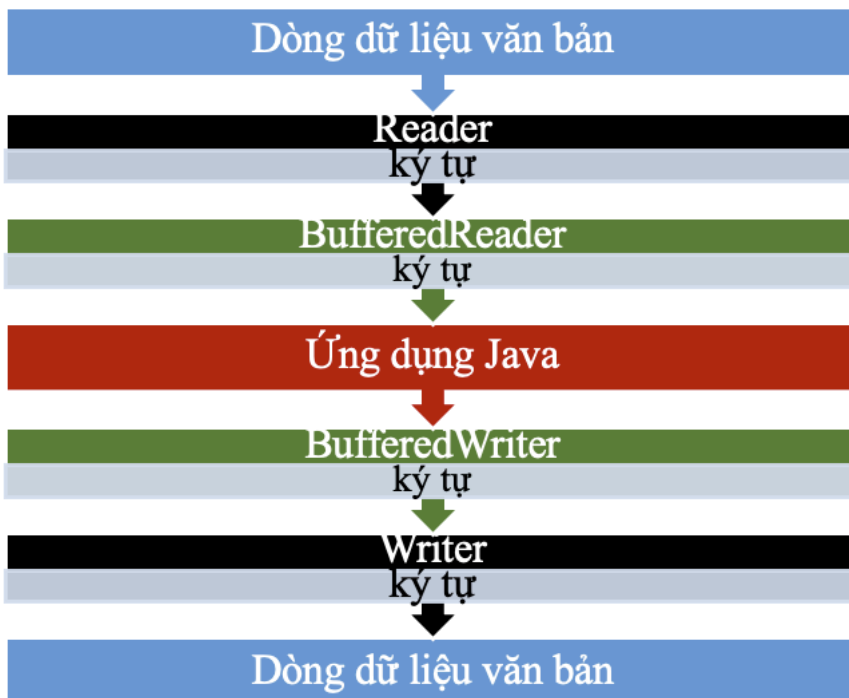
```

FileReader in = new FileReader ( "text.txt" );
int data = in.read();
while (data != - 1 ) {
System.out .printf( "%c" , data);
data = in.read();
}
in.close();

```

### 7.4.3. BufferedReader and BufferedWriter

Using the Readers and Writers mentioned above allows for easy manipulation of text data streams; however, to improve read and write performance, the BufferedReader and BufferedWriter classes can be used. These classes allow reading text data using buffers for more efficient input/output. Additionally, BufferedReader adds a `readLine()` method to read entire lines from the source text data.



and `BufferedReader` read/write methods

| Recording methods<br>by <b>BufferedWriter</b> | Reading methods<br>of <b>FileReader</b> | Describe                                         |
|-----------------------------------------------|-----------------------------------------|--------------------------------------------------|
| write()                                       | read()                                  | Read and write one or more characters.           |
| But                                           | readLine()                              | Read an entire line of characters from the data. |

For example, write a text file with the content "BufferedWriter":

```
FileWriter out = new FileWriter( "text.txt" );
BufferedWriter bw = new BufferedWriter(out);
bw.write( "BufferedWriter" );
bw.flush();
bw.close();
```

Read the aforementioned text file using **BufferedReader**. The reading speed will be similar for small files like this, but it will be significantly different for larger original files.

```
FileReader in = new FileReader( "text.txt" );
BufferedReader br = new BufferedReader(in);
String str = br.readLine();
while (str != null ) {
    System.out.printf ( str);
    str = br.readLine();
}
br.close();
```

## 7.5. READING AND WRITING OBJECTS

In Java, to read and write objects from a file, we can use **ObjectOutputStream** and **ObjectInputStream** objects. To use these objects, we need to pass an object of the **FileOutputStream** or **FileInputStream** class to perform the file reading or writing. However, the object to be written to the file must be implemented from the **Serializable** interface. For example, to write a student object to a file:

Create a Student class that implements the Serializable interface (Only when implementing from this interface can objects created from the Student class be written to a file).

```
class Student implements Serializable {  
    private int studentID ;  
    private String studentName ;  
  
    public Student( int studentID, String studentName) {  
        this.studentID = studentID ;  
        this . studentName = studentName;  
    }  
  
    public int getStudentID() {  
        return studentID ;  
    }  
  
    public void setStudentID( int studentID) {  
        this.studentID = studentID ;  
    }  
  
    public String getStudentName() {  
        return studentName ;  
    }  
  
    public void setStudentName(String studentName) {  
        this . studentName = studentName;  
    }  
}
```

Write code to write the object to a file:

```
public static void main(String[] args) {  
    ObjectInputStream in = null ;  
    ObjectOutputStream out = null ;  
    try {  
        out = new ObjectOutputStream( new BufferedOutputStream( new  
            FileOutputStream( "object.dat" )));  
    }  
}
```



```
        Student[] list = new Student[ 3 ];
list[ 0 ] = new Student( 1 , "Peter" );
list[ 1 ] = new Student( 2 , "David" );
list[ 2 ] = new Student( 3 , "Tommy" );

        out.writeObject(list);
out.close();
    } catch (Exception ex) {
ex.printStackTrace();
    }
}
```

It's important to note that if you want to prevent a property from being written to a file, use the keyword `transient` before each variable. This will cause Java to ignore the variable when **serializing** an object.

## 7.6. FILE CLASS

Files and folders on a computer are managed through the File class. To manage files or folders, you need to create a File object, for example:

```
File file = new File("myfile.txt");
```

The File class provides several methods:

- Boolean exist() - Checks if the object exists.
- Boolean canWrite() - Checks if write permission is granted.
- Boolean canRead() - Checks if read permission is granted.
- Boolean isFile() - Checks if it is a file.
- Boolean isDirectory() - Checks if it's a directory
- String getName() - Gets the file name
- String getAbsolutePath() - Gets the absolute path to the file.

## PRACTICE

## Lesson 1. Practice copying files.

This exercise will involve writing code to copy a file using the following methods:

- Do not use buffering.
- Use `BufferedInputStream`.
- Use Buffer with 4K resolution.
- Use buffers of different sizes.

### a. Do not use buffering.

Create and write the code for the `FileCopyNoBuffer` class:

```
import java.io.*;

public class FileCopyNoBuffer {
    public static void main(String[] args) {
        File fileIn;
        FileInputStream in = null ;
        FileOutputStream out = null ;
        long startTime, elapsedTime; // for speed benchmarking

        try {
            fileIn = new File( "a.jpg" );
            System.out.println( "File size is " + fileIn.length() + "
bytes" );
            in = new FileInputStream(fileIn);
            out = new FileOutputStream( "b.jpg" );

            startTime = System.nanoTime ();
            int byteRead;
            // Read an unsigned byte (0-255) and padded to
            32-bit int
            while ((byteRead = in.read()) != -1 ) {
                // Write the least significant bytes, drop the
                upper 3 bytes
                out.write(byteRead);
            }
        }
    }
}
```

```

elapsedTime = System. nanoTime () - startTime;
System. out .println( "Elapsed Time is " + (elapsedTime /
1000000.0 ) + " msec" );
} catch (IOException ex) {
ex.printStackTrace();
} finally { // always close the streams
    try {
        if (in != null ) in.close();
        if (out != null ) out.close();
    } catch (IOException ex) {
ex.printStackTrace();
}
}
}
}
}

```

Display the size and time required to copy the file:

```

The file size is 2906316 bytes
Elapsed Time is 10061.436241 msec

```

### **b. Use Buffered Streams.**

Create and write code for the **FileCopyBuffered** class:

```

import java.io.*;
public class FileCopyBuffered {
    public static void main(String[] args) {
File fileIn;
BufferedInputStream in = null ;
BufferedOutputStream out = null ;
        long startTime, elapsedTime;

        try {
fileIn = new File( "a.jpg" );
System. out .println( "File size is " + fileIn.length() + "
bytes" );
in = new BufferedInputStream( new FileInputStream(fileIn));

```

```

out = new BufferedOutputStream( new FileOutputStream( "b.jpg"
));
startTime = System. nanoTime ();
    int byteRead;
    while ((byteRead = in.read()) != - 1 ) {
out.write(byteRead);
}
elapsedTime = System. nanoTime () - startTime;
System. out .println( "Elapsed Time is " + (elapsedTime /
1000000.0 ) + " msec" );
} catch (IOException ex) {
ex.printStackTrace();
finally {
    try {
        if (in != null ) in.close();
        if (out != null ) out.close();
    } catch (IOException ex) { ex.printStackTrace(); }
}
}
}

```

Display the size and time required to copy the file:

The file size is 2906316 bytes

Elapsed Time is 90.529071 msec

### c. Use a 4K buffer.

Create and write code for the **FileCopyUserBuffer** class:

```

import java.io.*;
public class FileCopyUserBuffer {
    public static void main(String[] args) {
File fileIn;
FileInputStream in = null ;
FileOutputStream out = null ;
        long startTime, elapsedTime; // for speed benchmarking

        try {

```

```

fileIn = new File( "a.jpg" );
System.out.println( "File size is " + fileIn.length() + "
bytes" );
in = new FileInputStream(fileIn);
out = new FileOutputStream( "b.jpg" );
startTime = System.nanoTime ();

    byte [] byteBuf = new byte [ 4096 ]; // 4K buffer
    int numBytesRead;
    while ((numBytesRead = in.read(byteBuf)) != - 1 ) {
out.write(byteBuf, 0 , numBytesRead);
}
elapsedTime = System.nanoTime () - startTime;
System.out.println( "Elapsed Time is " + (elapsedTime /
1000000.0 ) + " msec" );
} catch (IOException ex) {
ex.printStackTrace();
} finally { // always close the streams
    try {
        if (in != null ) in.close();
        if (out != null ) out.close();
    } catch (IOException ex) { ex.printStackTrace(); }
}
}
}
}

```

Display the size and time required to copy the file:

The file size is 2906316 bytes

Elapsed Time is 4.243062 msec

Rewrite the program in the **FileCopyUserBuffer** class , using different buffer sizes:

```

try {
    int [] bufSizeKB = { 1 , 2 , 4 , 8 , 16 , 32 , 64 , 256 ,
1024 }; // print KB
    fileIn = new File( "a.jpg" );

```

```

System. out .println( "File size is " + fileIn.length() + "
bytes" );

    for ( int i = 0 ; i < bufSizeKB. length ; i++) {
in = new FileInputStream(fileIn);
out = new FileOutputStream( "b.jpg" );
startTime = System. nanoTime ();
        int bufSize = bufSizeKB[i] * 1024 ;
        byte [] byteBuf = new byte [bufSize]; // buffer
        int numBytesRead;
        while ((numBytesRead = in.read(byteBuf)) != - 1 ) {
out.write(byteBuf, 0 , numBytesRead);
}
elapsedTime = System. nanoTime () - startTime;
System. out .println(bufSizeKB[i] + "KB Elapsed Time is " +
(elapsedTime / 1000000.0 ) + " msec" );
}
} catch (IOException ex) {
ex.printStackTrace();
} finally { // always close the streams
    try {
        if (in != null ) in.close();
        if (out != null ) out.close();
    } catch (IOException ex) {
ex.printStackTrace();
}
}
}

```

Please show the program's output and indicate what buffer size will result in the fastest file copying time.

## Lesson 2. Navigating the contents of a folder.

Create a ListDirectoryRecursive class. Write a listRecursive() method.

```

public static void listRecursive(File dir) {
    if (dir.isDirectory()) {
File[] items = dir.listFiles();

```

```

        if (items != null ) {
            for (File item : items) {
System. out .println(item.getAbsolutePath());
                if (item.isDirectory())
                    listRecursive (item);
            }
        }
    }
}

```

The main program for listing the contents of a directory:

```

public static void main(String[] args) throws IOException {
File dir = new File( "C: \\ Windows" );
listRecursive(dir);
}

```

Implement the program:

```

C:\Windows\actsetup.log
C:\Windows\addins\
C:\Windows\addins\FXSEXT.ecf
C:\Windows\AppCompat
...

```

### Lesson 3. Reading data using a scanner.

Create a data file named **test.txt** with the following content:

```

1 fish 2 fish red fish blue fish

```

Data like the one above can be read using a scanner as follows:

```

File text = new File( "test.txt" );
Scanner scanner = new Scanner(text);
while ( scanner .hasNextLine()) {
    String line = scanner .nextLine();
    System . out .println( line );
}

```

```
}
```

Results after running:

```
1 fish 2 fish red fish blue fish
```

The Scanner object provides a method called `useDelimiter` that makes it easy to separate source data, for example, by spaces.

```
File text = new File( "test.txt" );
Scanner scanner = new Scanner( text ).useDelimiter( " \\ s " );
//space;;

System . out .println( scanner .nextInt());
System . out .println( scanner .next());
System . out .println( scanner .nextInt());
System . out .println( scanner .next());
scanner.close ();
```

Results after running:

```
1
fish
2
fish
```

## Lesson 4. Reading data online

Read COVID information from the API and save it to a file named `COVIDSummary.txt`.

```
String url = "https://api.covid19api.com/summary" ;
String fileName = "Covid Summary.txt" ;
BufferedInputStream in = new BufferedInputStream( new
URL(url) .openStream());
FileOutputStream fileOutputStream = new
FileOutputStream(fileName);
byte dataBuffer[] = new byte [ 1024 ];
int bytesRead;
```



```
while ((bytesRead = in.read(dataBuffer, 0 , 1024 )) != - 1 ) {
    fileOutputStream.write(dataBuffer, 0 , bytesRead);
}
```

The data that will be saved will be as follows:

```
{"Message":"Caching in
progress","Global":{"NewConfirmed":0,"TotalConfirmed":0,"NewDeaths":0,"TotalDeaths":0,"NewRecovered":0,"TotalRecovered":0,"Date":"0001-01-01T00:00:00Z"},"Date":"0001-01-01T00:00:00Z"}
```

Another example: reading a text file stored over the network:

```
String url = "https://www.w3schools.com/js/json_demo.txt";
BufferedReader br = new BufferedReader( new InputStreamReader(
new URL( url ).openStream()));
String str = br.readLine ();
while ( str != null) {
    System.out.printf ( str ) ;
    str = br.readLine ();
}
br .close();
```

Data to be printed to the screen:

```
{ "name":"John", "age":31, "pets":[ { "animal":"dog",
"name":"Fido" }, { "animal":"cat", "name":"Felix" }, {
"animal":"hamster", "name":"Lightning" } ]}
```

## QUIZZES

|                                                                      |  |
|----------------------------------------------------------------------|--|
| The classes for performing input/output are declared in the package: |  |
| java.io                                                              |  |
| java.stream                                                          |  |
| java.file                                                            |  |
| java.in and java.out                                                 |  |

|                            |  |
|----------------------------|--|
| An InputStream is used to: |  |
|----------------------------|--|

|                       |  |
|-----------------------|--|
| Reading binary data   |  |
| Binary data recording |  |
| Reading text data     |  |
| Record text data      |  |

|                            |  |
|----------------------------|--|
| BufferedReader is used to: |  |
| Reading binary data        |  |
| Binary data recording      |  |
| Reading text data          |  |
| Record text data           |  |

|                               |  |
|-------------------------------|--|
| FileOutputStream is used for: |  |
| Reading data in a stream.     |  |
| Reading binary data           |  |
| Record text data              |  |
| Write data to a file          |  |

|                                                                         |  |
|-------------------------------------------------------------------------|--|
| To write a double number to a file, which class method should be used ? |  |
| DataOutputStream                                                        |  |
| Buffered Input Stream                                                   |  |
| BufferDataStream                                                        |  |
| NumberStream                                                            |  |

|                                                            |  |
|------------------------------------------------------------|--|
| To improve performance and input/output speed:             |  |
| Binary data types should be used.                          |  |
| Buffered classes should be used.                           |  |
| Use primitive data.                                        |  |
| Multiple processes should be created for input and output. |  |

|                                        |  |
|----------------------------------------|--|
| If an object wants to write to a file: |  |
|----------------------------------------|--|

|                                                                              |  |
|------------------------------------------------------------------------------|--|
| The class that creates the object must inherit from the Serializable class.  |  |
| The class that creates the object must inherit from the Writable class.      |  |
| The class that creates the object must implement the Serializable interface. |  |
| The class that creates the object must implement the Writable interface.     |  |

|                                          |  |
|------------------------------------------|--|
| The File class can be used to            |  |
| Reading data from a file                 |  |
| Get information about files and folders. |  |
| Write data to a file.                    |  |
| Scan and search for data from files.     |  |

|                                                                                    |  |
|------------------------------------------------------------------------------------|--|
| To access and exit data over the network:                                          |  |
| It is mandatory to use the classes in java.nio.                                    |  |
| You must use the Scanner object.                                                   |  |
| The OutputStream class needs to be used.                                           |  |
| An InputStream needs to be opened that connects to the data source on the network. |  |

|                                                                                                                                                                                                                                                                                                                                                                                                    |  |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| <p>What is the output of the following code snippet ?</p> <pre> File someFile = new File( "test.txt" ); int aCount = 0 ; try (FileReader fr = new FileReader(someFile);     BufferedReader br = new BufferedReader( fr )) {     int ch ;     while ( ( ch = br .read()) != - 1 ) {         if ( ch == 'i' ) {             aCount ++;         }     }     System . out .println( aCount ); } </pre> |  |
| Print the number of 'i's in the test file.                                                                                                                                                                                                                                                                                                                                                         |  |

|                                           |  |
|-------------------------------------------|--|
| Print the letter i                        |  |
| Copy the file test.txt to Buffer memory.  |  |
| Write the letter 'i' in the file test.txt |  |

|                                                                                                                                                                                                                                                                                                                                   |  |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| <p>What is the output of the following code snippet ?</p> <pre> try {     Scanner scanner = new Scanner( new File( "test.txt" ));     while ( scanner .hasNextLine()) {         String line = scanner .nextLine();         System . out .println( line );     } } catch (FileNotFoundException e) { e.printStackTrace(); } </pre> |  |
| Read the file test.txt and print it to the screen.                                                                                                                                                                                                                                                                                |  |
| Count the number of lines in the test.txt file.                                                                                                                                                                                                                                                                                   |  |
| Check for the existence of the file test.txt                                                                                                                                                                                                                                                                                      |  |
| Scan and count the number of characters in the file test.txt                                                                                                                                                                                                                                                                      |  |

## PRACTICE EXERCISES

### Lesson 1. Writing a string to a file using OutputStream

Write a Java program to write a string of data to a file using the OutputStream class in the java.io package.

### Lesson 2. Using ObjectInputStream and ObjectOutputStream

Create a User object with the following properties: id (int type), name (String type), age (int type).

Store the user's information in the file user.dat using ObjectOutputStream. Read the user's information from the file user.dat using ObjectInputStream and display the information on the console.

### **Lesson 3. Count words in a text file**

Write a Java program to read the contents of a text file and count the number of words in that file using the Reader and Writer classes.

### **Lesson 4. Creating files and folders**

Write a Java program to create a new directory and a new file within it. Name the directory "My Directory," and name the file using the current date and time.

### **REFERENCES**

- [1] Core Java: Fundamentals (2021) , Cay Horstmann (Oracle Press Java).
- [2] Head First Java: A Brain-Friendly Guide (2022), Kathy Sierra, O'Reilly Media.
- [3] Java OOP Done Right: Create object oriented code you can be proud of with modern Java Paperback (2019), Mr Alan Mellor, Mellor Books.
- [4] Murach's Java Programming (5th Edition) (2017), Joe Murach, Mike Murach & Associates.
- [5]. Java for Absolute Beginners Learn to Program the Fundamentals the Java 9+ Way.
- [6]. Modern Java Recipes: Simple Solutions to Difficult Problems in Java 8 and 9 (2017), by Ken Kousen, O'Reilly Media.
- [7] Effective Java (2018), Joshua Bloch, Addison-Wesley Professional.